Esercizio Laboratorio 2:

Progetto: "Battle Royale - Safe Zone Controller"

Lo Scenario: Stai programmando il server centrale di un gioco multiplayer in stile *Battle Royale* (tipo Fortnite o Warzone, ma testuale). Nel gioco, i giocatori si muovono continuamente in una mappa. Ogni tot secondi, una "Safe Zone" (Zona Sicura) si restringe. Chi si trova fuori dalla Safe Zone prende danno finché non muore.

Per gestire questo sistema servono due canali:

1. **TCP (Porta 8080 - "Il Canale di Stato"):** Gestisce la connessione iniziale (Login), l'assegnazione dell'ID, e gli eventi critici di gioco (ricevere danno, morire, vincere). Questo canale deve essere affidabile.
2. **UDP (Porta 8081 - "Il Canale di Movimento"):** I giocatori "spammano" le loro coordinate X e Y al server più velocemente possibile. Se un pacchetto di movimento si perde per lag, non importa, arriverà quello del decimo di secondo successivo.

---

1. Il Server (Game Coordinator)

Il server deve mantenere lo stato di tutti i giocatori in una struttura dati globale in RAM (es. un array di struct o una lista), protetta rigorosamente da un `pthread_mutex_t`. La struct del singolo giocatore dovrà contenere: `ID`, `Socket_TCP`, `Health` (parte da 100), `Pos_X`, `Pos_Y`, `is_alive`.

Il Server è composto da **3 anime (Thread separati)**:

- **Thread TCP (Accettazione & Login):** Sta in `accept()`. Quando un giocatore si connette, gli assegna un ID univoco, lo inserisce nella lista globale con `Health = 100`, e gli risponde con un pacchetto TCP `GAME_JOINED` contenente il suo ID. Fatto ciò, il

thread TCP si mette in ascolto bloccante solo per capire se il giocatore si disconnette ("rage quit").

- **Thread UDP (Tracciamento Movimenti):** Un singolo thread bloccato su una `recvfrom()`. Riceve a raffica i pacchetti `MOVEMENT` (con ID, X e Y) da tutti i giocatori. Quando riceve un pacchetto, cerca l'ID nella lista globale e **aggiorna le sue coordinate X e Y**.
- **Thread "Game Loop" (La vera sfida dell'esame):** Questo è un thread speciale che gira in background con un ciclo infinito e uno `sleep(5)` (si sveglia ogni 5 secondi). Quando si sveglia:
  1. Calcola o stabilisce una "Safe Zone" (per semplicità dell'esercizio: un quadrato, es. da X=20 a X=80, e da Y=20 a Y=80).
  2. Blocca il Mutex e scorre tutti i giocatori vivi.
  3. Controlla le loro ultime coordinate X e Y (aggiornate dal thread UDP).
  4. **Se il giocatore è FUORI dalla Safe Zone:** Il server gli infligge 20 danni. Usa il `Socket_TCP` di quel giocatore per mandargli un pacchetto `DAMAGE_TAKEN`.
  5. Se la vita del giocatore scende a 0, gli manda un pacchetto TCP `ELIMINATED`, chiude il suo socket e lo rimuove dalla partita.

2. Il Client (Il Giocatore)

All'avvio, il client non ha bisogno di parametri.

1. **Fase di Connessione (TCP):** Si connette alla porta 8080 del server. Aspetta di ricevere il pacchetto `GAME_JOINED` per scoprire qual è il suo `ID_Giocatore`.
2. **Sdoppiamento (Multi-Thread):**
   - **Thread A (Motore di Movimento - UDP):** Gira in un `while(1)`. Ogni mezzo secondo (`usleep(500000)`), genera coordinate X e Y

casuali (es. tra 0 e 100). Costruisce un pacchetto `MOVEMENT` con il suo ID e lo spara al server via UDP sulla porta 8081.

- **Thread B (Interfaccia Eventi - TCP):** Gira bloccato su una `recv()` del socket TCP.
  - Se riceve `DAMAGE_TAKEN`, stampa: `[!!!] SEI FUORI DALLA SAFE ZONE! Hai preso danno. Vita rimanente: X`.
  - Se riceve `ELIMINATED`, stampa: `[X_X] SEI MORTO! Fine partita.`, chiude tutto ed esce dal programma con `exit(0)`.

La "Trappola da Esame" (A cosa stare attenti)

La complessità qui è la **Concorrenza sulla Memoria (Data Race)**. Hai il *Thread UDP* che scrive costantemente dentro la struct dei giocatori per aggiornare X e Y. Contemporaneamente, hai il *Thread Game Loop* che legge quelle stesse X e Y per calcolare i danni e modificare la variabile `Health`. Se non metti un `pthread_mutex_lock()` prima di leggere/scrivere le coordinate e non fai `pthread_mutex_unlock()` subito dopo, il programma andrà in *Segmentation Fault* o calcolerà danni a caso.

Struttura Pacchetto Consigliata (`Protocol.h`)

```
#define PORT_TCP 8080
#define PORT_UDP 8081

// Tipi di pacchetto
#define TYPE_GAME_JOINED 1
#define TYPE_MOVEMENT 2
#define TYPE_DAMAGE_TAKEN 3
#define TYPE_ELIMINATED 4

typedef struct {
    int type;           // Tipo di messaggio
    int player_id;      // Assegnato dal server
```

```
    int pos_x;          // Usato in UDP
    int pos_y;          // Usato in UDP
    int health_left;    // Usato dal server per comunicare la
vita dopo il danno
} GamePacket;
```